

LAB – SOCKET PROGRAMMING

Student Details:

Name: Louis Muiyuro

Admission Number: 169275

Unit: Distributed Systems (ICS 4104)

Title: Client-Server Communication and Synchronisation Using Java Sockets

Group Number: 14

Theory:

Programs do not know how to talk to network hardware directly. A socket is used by the operating system to send and receive data over the network, without a program needing to know anything about the hardware underneath. A socket is identified by two pieces of information: the IP address, which identifies the specific device to connect to, and the port number, which identifies the specific program on that device to communicate with.

TCP Sockets

This lab illustrates a TCP socket, which guarantees three things:

- Messages arrive in order.
- Messages must arrive at all (losing a message, e.g. in a chat application, is unacceptable).
- Each side can detect when the other side has disconnected.

ServerSocket vs Socket

ServerSocket does not send or receive messages. Its only job is to sit at a chosen port and listen for incoming connection requests — it lacks an InputStream or OutputStream, so it cannot itself read or write data. The actual conversation happens over a Socket object, obtained either by the server accepting a connection or by the client initiating one:

```
ServerSocket ss = new ServerSocket(5555);

Socket s = ss.accept();                                // server side: accept a client

Socket s = new Socket("127.0.0.1", 5555);              // client side: connect to
server
```

Why the Server Must Start First

The client's very first action is to reach out and connect. If nobody is listening on the other end when it does so, the connection attempt fails immediately with a connection-refused error. The server must therefore be running and listening on its port before any client attempts to connect.

Aim:

To develop a distributed client-server application using Java Sockets, and to demonstrate synchronised, ordered message exchange between a single client and server over a TCP connection.

Algorithm:

(a) Server (MyServer.java)

- Create a ServerSocket bound to a fixed port (5555) and wait for a client to connect.
- Accept the incoming connection, producing a Socket for communication with that client.
- Open an input stream (to receive from the client) and an output stream (to send to the client).
- In a loop: read a line from the client and print it, then read a line typed at the keyboard and send it back to the client.
- If either side sends the message "bye", end the loop.
- Close the socket and streams, and print that the connection has been terminated.

(b) Client (MyClient.java)

- Create a Socket connecting to the server's IP address and port (127.0.0.1:5555).
- Open an input stream (to receive from the server) and an output stream (to send to the server).
- In a loop: read a line typed at the keyboard and send it to the server, then read the server's reply and print it.
- If either side sends the message "bye", end the loop.
- Close the socket and streams, and print that the connection has been terminated.

Implementation:

The lab was implemented using Java in a Windows terminal environment. Two separate programs were run as independent processes in separate terminal windows: `MyServer.java`, which listens for and responds to a client, and `MyClient.java`, which initiates the connection and exchanges messages with the server.

(a) Software Requirements:

- Java Development Kit (JDK)
- Java Compiler (javac)
- Java Runtime (java)
- Two terminal windows (one for the server, one for the client)

(b) Compilation:

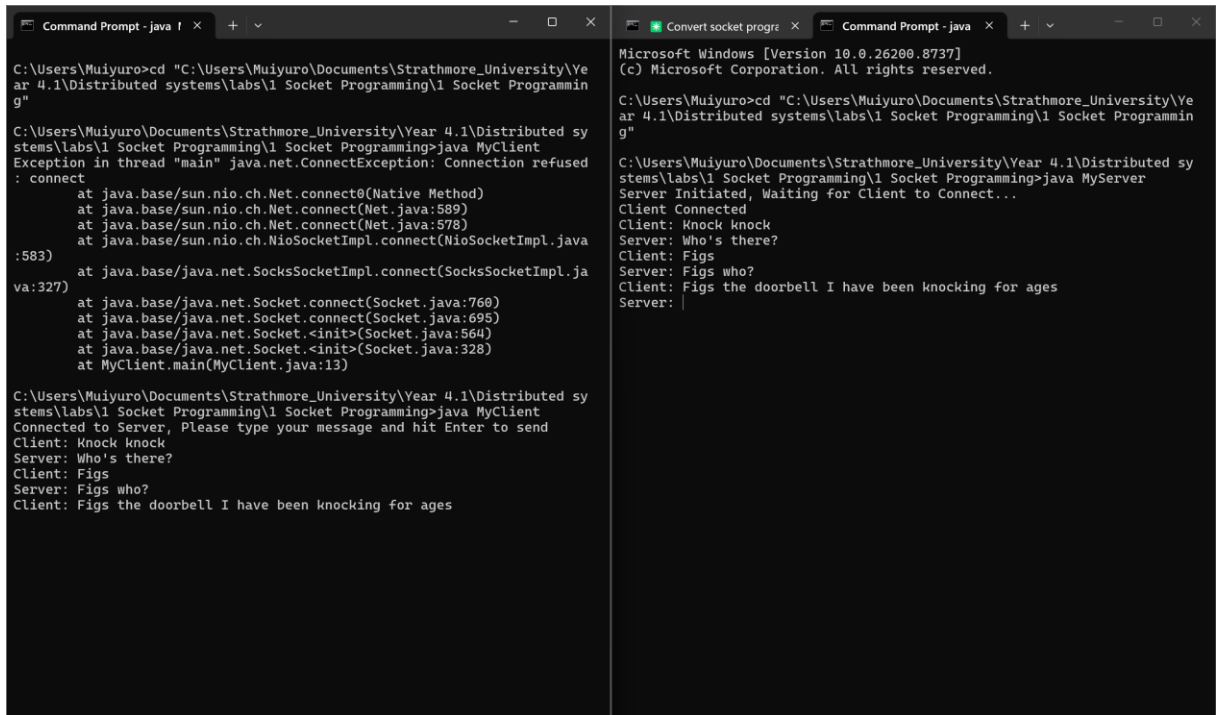
Both source files were compiled from the project folder using:

```
javac MyServer.java MyClient.java
```

This produced `MyServer.class` and `MyClient.class`.

(c) Execution:

The client was first run on its own, before the server had been started, to confirm the server-first requirement described in the Theory section above — it failed immediately with a connection-refused error since nothing was listening on port 5555. The server was then started in a second terminal, after which the client connected successfully and the two exchanged a short knock-knock joke, alternating messages between Client and Server until the conversation ended.



```
C:\Users\Muiyuro>cd "C:\Users\Muiyuro\Documents\Strathmore_University\Year 4.1\Distributed systems\labs\1 Socket Programming\1 Socket Programming"
C:\Users\Muiyuro\Documents\Strathmore_University\Year 4.1\Distributed systems\labs\1 Socket Programming>java MyClient
Exception in thread "main" java.net.ConnectException: Connection refused
: connect
    at java.base/sun.nio.ch.Net.connect0(Native Method)
    at java.base/sun.nio.ch.Net.connect(Net.java:589)
    at java.base/sun.nio.ch.Net.connect(Net.java:578)
    at java.base/sun.nio.ch.NioSocketImpl.connect(NioSocketImpl.java:583)
    at java.base/java.net.SocksSocketImpl.connect(SocksSocketImpl.java:327)
    at java.base/java.net.Socket.connect(Socket.java:760)
    at java.base/java.net.Socket.connect(Socket.java:695)
    at java.base/java.net.Socket.<init>(Socket.java:564)
    at java.base/java.net.Socket.<init>(Socket.java:328)
    at MyClient.main(MyClient.java:13)
C:\Users\Muiyuro\Documents\Strathmore_University\Year 4.1\Distributed systems\labs\1 Socket Programming>java MyClient
Connected to Server, Please type your message and hit Enter to send
Client: Knock knock
Server: Who's there?
Client: Figs
Server: Figs who?
Client: Figs the doorbell I have been knocking for ages

Microsoft Windows [Version 10.0.26200.8737]
(c) Microsoft Corporation. All rights reserved.
C:\Users\Muiyuro>cd "C:\Users\Muiyuro\Documents\Strathmore_University\Year 4.1\Distributed systems\labs\1 Socket Programming\1 Socket Programming"
C:\Users\Muiyuro\Documents\Strathmore_University\Year 4.1\Distributed systems\labs\1 Socket Programming>java MyServer
Server Initiated, Waiting for Client to Connect...
Client Connected
Client: Knock knock
Server: Who's there?
Client: Figs
Server: Figs who?
Client: Figs the doorbell I have been knocking for ages
Server: |
```

Limitation and Potential Improvement:

This implementation has a single-client limitation: once one client disconnects, the server closes completely and exits — it does not wait for a second client to connect. A straightforward improvement would be to wrap the accept call in a loop, so the server keeps listening for new clients after each conversation ends, rather than terminating after the first one:

```
while (true) {

    Socket s = ss.accept();

    // handle this client's conversation

}
```

On its own this loop only allows one client conversation to be handled at a time (the server would still need to finish with one client before accepting the next). Supporting multiple clients simultaneously would additionally require handling each accepted connection on its own thread.

Conclusion:

This lab demonstrated the fundamentals of socket-based distributed communication in Java: how a `ServerSocket` listens for connections while a `Socket` carries the actual conversation, why the server process must be started before any client can connect, and how TCP guarantees ordered, reliable delivery of messages between two communicating processes. Running the client before and after starting the server confirmed the server-first requirement in practice, and the successful message exchange verified that synchronised client-server communication was achieved. The single-client limitation observed also highlighted the need for looped or multi-threaded connection handling in any server expected to support more than one client.